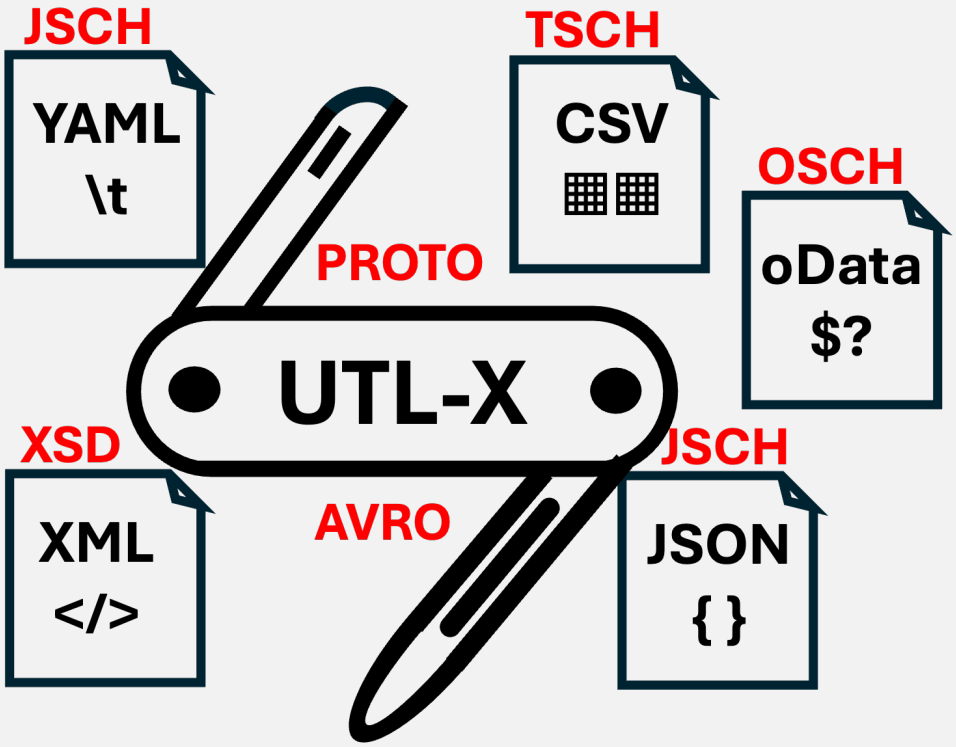
GLOMIDCO

UTLXe

Production Transformation Engine



Deployment and Operations Guide

Running UTLXe on Azure Container Apps — From First Deploy to Production

Ir. Marcel A. Grauwen

First Edition — 2026

UTLXe on Azure — Deployment and Operations Guide

Published by GLOMIDCO B.V., The Netherlands

First edition, 2026.

UTL-X is open source software. The language specification, CLI tool, and standard library are freely available at `https://github.com/grauwen/utl-x`.

This is a companion guide to *UTL-X: One Language, All Formats* (ISBN 978-90-819728-1-9), which covers the complete language specification, standard library, and format system.

Typeset with Typst in New Computer Modern.

Table of Contents

Part I

Getting Started

Why UTLXe, and how to get started

1 Why UTLXe?

Every integration project has the same problem: data arrives in one format and needs to leave in another. An order from Dynamics 365 is JSON. The Peppol e-invoicing network expects UBL XML. A warehouse system sends CSV. An IoT platform produces YAML. The business logic is often simple — rename a field, calculate a total, filter a list — but the plumbing to connect these systems is not.

This chapter explains where UTLXe fits in an Azure middleware architecture, what problems it solves, and why those problems are harder than they appear.

1.1 The Transformation Problem

Consider a common Azure integration: Dynamics 365 Business Central publishes sales orders to Azure Service Bus. A downstream system consumes them and expects a different structure.

Without UTLXe, you write the transformation in one of these places:

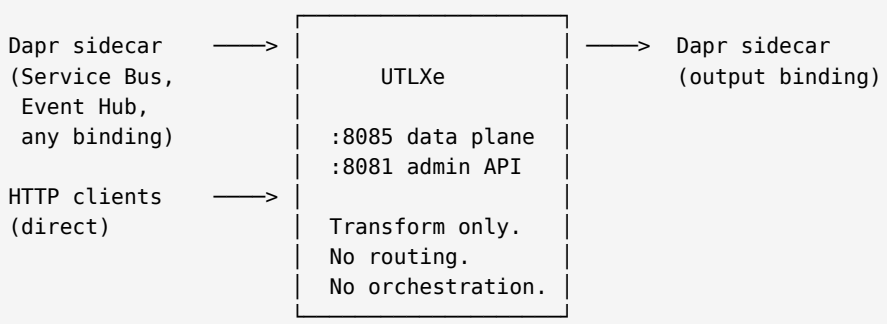
Approach	**How**	**Problem**
Azure Function	C# or Python code that parses JSON, builds output, serializes	Business logic buried in infrastructure code. Every mapping change requires a code deployment.
Logic App	Visual designer with built-in transforms	Limited expression language. Complex mappings become unreadable. No version control for visual flows.
API Management policy	Liquid templates or XSLT in the gateway	Mixing transformation with routing. Templates are hard to test. No debugging.
In the producer	D365 emits the target format directly	Tight coupling. Producer must know every consumer's format. Changes ripple upstream.
In the consumer	Downstream system accepts the raw format	Same coupling problem in reverse. Every consumer must handle every producer's format.

All of these approaches scatter transformation logic across the architecture. When the Peppol specification changes, or a new trading partner requires a different XML dialect, or the warehouse system switches from CSV to JSON, someone has to find the right Azure Function, Logic App, or XSLT template and modify it.

1.2 What the UTLXe Azure Offering Does

UTLXe is a dedicated transformation engine that runs as a container on Azure Container Apps. Its only job is to transform data — parse an incoming message, apply mapping logic, and produce output in the required format.

UTLXe exposes two HTTP ports. That is its entire external interface:

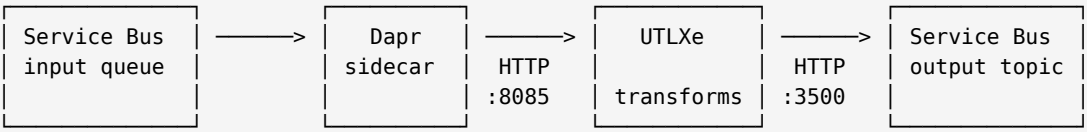


Port 8085 is the data plane — messages in, results out. Port 8081 is the admin API — upload transformations, check health, view metrics. UTLXe never connects to Azure services directly. It receives and sends HTTP. The connection to messaging infrastructure is handled by a Dapr sidecar.

The following sections show each connection scenario with a concrete example.

1.2.1 Scenario 1: Azure Service Bus via Dapr

The most common pattern. Messages arrive on a Service Bus queue, get transformed, and are forwarded to an output queue or topic.

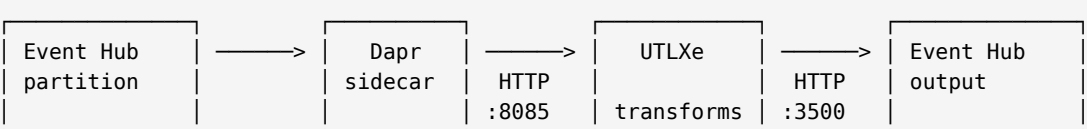


Dapr connects to Service Bus using a YAML component definition that you configure in the Azure Portal or via Azure CLI. When a message arrives on the queue, Dapr calls UTLXe at `POST http://utlxe:8085/api/dapr/input/orders-in`. UTLXe transforms the message and sends the result back to Dapr, which delivers it to the output queue.

No Azure SDK, no connection code, no retry logic in UTLXe. Dapr handles connection management, retries, and dead-letter routing. Chapter 4 walks through the complete setup step by step — from creating the Service Bus namespace to configuring the Dapr component in the Azure Portal.

1.2.2 Scenario 2: Azure Event Hub via Dapr

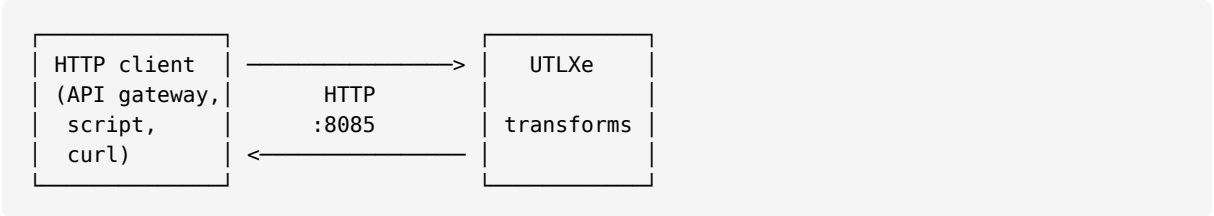
Same pattern, different Dapr component type. Event Hub is used for high-throughput streaming scenarios.



The Dapr component type changes to `bindings.azure.eventhubs`, but the UTLXe side is identical — it does not know or care whether the message came from Service Bus or Event Hub. Dapr abstracts the transport. The step-by-step setup is covered in Chapter 4.

1.2.3 Scenario 3: Direct HTTP (no Dapr)

For request/response patterns — API gateways, batch scripts, testing — clients call UTLXe directly:




```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"orderNumber":"12345","amount":200.00}' \
  http://utlxe-ingress:8085/api/transform/invoice-to-ubl
```

No Dapr, no Service Bus. A direct HTTP call, a transformed response. Useful for synchronous integrations, testing, and API format conversion.

1.2.4 The Transformation

In all three scenarios, the transformation is the same `.utlx` file:

```
%utlx 1.0
input json
output xml
---
{
  Invoice: {
    ID: concat("INV-", $input.orderNumber),
    IssueDate: formatDate(now(), "yyyy-MM-dd"),
    BuyerParty: { Name: $input.customer.name },
    Total: round($input.amount * 1.21, 2)
  }
}
```

The transformation does not contain any reference to Service Bus, Event Hub, or HTTP. It expresses only the mapping. The transport is a deployment concern, not a transformation concern.

This pattern applies wherever data crosses a format or schema boundary: e-invoicing (Peppol, UBL), API format conversion, IoT sensor normalization, ERP migration, and B2B partner integration.

UTLXe is capable of much more than what this Azure offering exposes — including gRPC transport, protobuf-based language wrappers for .NET, Go, and Python, Kafka integration, and pipeline chaining. For the full capabilities, see the UTLXe documentation at `https://github.com/grauwen/utl-x`. This book covers only the Azure Marketplace deployment.

1.3 The Cost of Embedded Transformation

To understand why a dedicated engine matters, consider what happens when transformation logic is embedded in an Azure Function:

```
// OrderToInvoice.cs — 180 lines
public static async Task<IActionResult> Run(
    [ServiceBusTrigger("orders")] string message,
    [ServiceBus("invoices")] IAsyncCollector<string> output)
{
    var order = JsonConvert.DeserializeObject<Order>(message);
    var invoice = new Invoice {
        ID = "INV-" + order.OrderNumber,
        IssueDate = DateTime.UtcNow.ToString("yyyy-MM-dd"),
        // ... 150 more lines of mapping logic
    };
    var xml = new XmlSerializer(typeof(Invoice))...
    await output.AddAsync(xml);
}
```

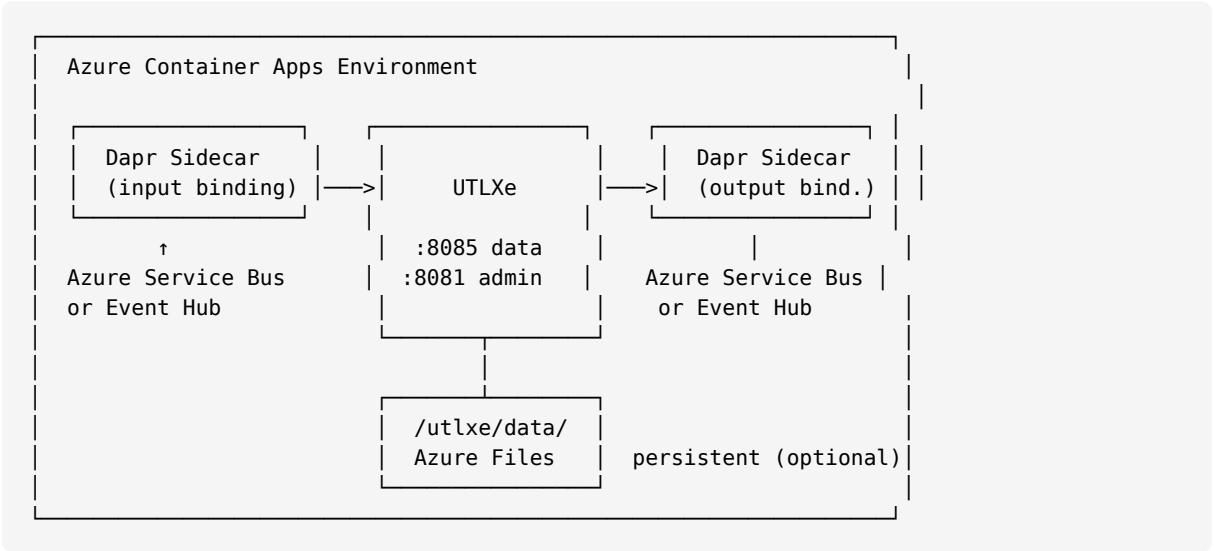
This works, but:

- The mapping logic is mixed with the infrastructure (Service Bus trigger, serialization, error handling).
- Testing requires a running Service Bus emulator or mock.
- Changing one field mapping requires a full C# build and deployment.
- The developer needs to know C#, the Azure Functions SDK, and the XML serialization API — in addition to the business mapping rules.

With UTLXe, the same mapping is twelve lines of `.utlx`. No build step, no SDK, no deployment pipeline for the container. Upload the file and it works. Change a field, re-upload, zero downtime.

1.4 How UTLXe Runs on Azure

UTLXe is available as a pre-built container on the Azure Marketplace. You deploy it like any other Container App — no custom Docker image needed.



- **Azure Container Apps** runs the container and handles scaling, health checks, networking, and TLS.
- **Dapr sidecar** connects to Azure messaging services. Configured with YAML, no code.
- **UTLXe** transforms messages. Configured with `.utlx` files, no code.
- **Azure Files** (optional) persists uploaded transformations across container restarts.

1.5 The Deployment Workflow

Unlike Azure Functions or Logic Apps, UTLXe separates the container from the transformations. The container is deployed once from the Marketplace. The transformations are deployed independently via the Admin API:

1. **Deploy the container** from the Azure Marketplace (once).
2. **Upload transformations** via `POST /admin/bundle` or individual `POST /admin/transformations/{name}`.
3. **Test** each transformation with sample input via `POST /admin/transformations/{name}/test`.
4. **Send traffic** to the data plane on port 8085.
5. **Update transformations** at any time — upload a new version, zero downtime.

No Docker builds, no container restarts, no CI/CD pipeline for infrastructure. The pipeline only ships `.utlx` files.

1.6 What UTLXe Does Not Do

UTLXe is a transformation engine, not an integration platform. It deliberately does not route messages, orchestrate workflows, store data, or manage connections. Dapr handles messaging. Logic Apps handle orchestration. Azure Container Apps handles infrastructure. UTLXe handles transformation — and only transformation.

This narrowness is intentional. A tool that does one thing well composes better than a tool that does everything adequately.

1.7 When to Use UTLXe

Use UTLXe when:

- Data arrives in one format and must leave in another.
- Transformation logic changes more often than infrastructure.
- Multiple team members need to read and modify mappings.
- You want version-controlled, testable, reviewable transformation logic.
- You need zero-downtime updates for transformation rules.

Use Azure Functions or Logic Apps instead when:

- The integration involves orchestration (multi-step, conditional branching, human approval).
- The transformation is trivial (rename one field) and doesn't justify a dedicated engine.
- You need to call external APIs as part of the transformation (UTLXe is stateless and does not make outbound API calls during transformation).

2 Quick Start

This chapter gets you from zero to a running transformation in under fifteen minutes. You will deploy UTLXe from the Azure Marketplace, upload a transformation, test it, and send your first real message.

2.1 What You Get

When you deploy UTLXe from the Azure Marketplace, Azure creates a Container App running the UTLXe production engine. The container exposes two ports:

- **Port 8085** — the data plane. Your applications send messages here for transformation. This port is exposed via the Container App ingress.
- **Port 8081** — the admin and health port. You manage transformations here. This port stays internal to your VNet.

The container starts empty — no transformations are loaded. You deploy them via the Admin API.

2.2 Deploy from the Azure Marketplace

1. Open the Azure Portal and search for "UTLXe" in the Marketplace.
2. Select your plan:
   - **Starter** (2 GB) — suitable for messages up to 50 KB, development and light production workloads.
   - **Professional** (4 GB) — suitable for messages up to 200 KB, production workloads with higher throughput.
3. Configure the deployment:
   - Resource group and region.
   - Container App name.
   - Persistent storage toggle — enable this if you want transformations to survive container restarts.
4. Click **Create**. The deployment takes approximately two minutes.

Once complete, note the Container App's internal IP (for the admin port) and the ingress URL (for the data plane).

2.3 Set the Admin Key

Before you can manage transformations, set the admin API key. This key protects the management endpoints from unauthorized access.

```
az containerapp secret set \
  -n utlxe -g myResourceGroup \
  --secrets admin-key=my-secret-key-here

az containerapp update \
  -n utlxe -g myResourceGroup \
  --set-env-vars UTLXE_ADMIN_KEY=secretref:admin-key
```

2.4 Verify the Deployment

Check that the container is running:

```
curl -s http://<internal-ip>:8081/health
```

```
{"status": "UP", "transformations": 0, "ready": false}
```

The engine is alive but has no transformations yet. The `ready: false` flag tells Kubernetes not to route traffic to this container.

2.5 Write Your First Transformation

Create a file called `hello.utlx`:

```
%utlx 1.0
input json
output json
---
{
  greeting: concat("Hello, ", $input.name, "!"),
  timestamp: now()
}
```

This transformation takes a JSON object with a `name` field and produces a greeting with a timestamp.

2.6 Upload It

```
curl -X POST \
  -H "X-Admin-Key: my-secret-key-here" \
  -F "source=@hello.utlx" \
  http://<internal-ip>:8081/admin/transformations/hello
```

```
{
  "status": "deployed",
  "name": "hello",
  "strategy": "COMPILED",
  "config": "defaults",
  "compiled_in_ms": 48
}
```

The transformation was compiled in 48 milliseconds and is ready to process messages. Check the health endpoint again:

```
{"status": "UP", "transformations": 1, "ready": true}
```

Now `ready: true` — the container will accept traffic.

2.7 Test It

Before routing real traffic, verify the transformation works with sample input:

```
curl -X POST \
  -H "X-Admin-Key: my-secret-key-here" \
  -H "Content-Type: application/json" \
  -d '{"name": "World"}' \
  http://<internal-ip>:8081/admin/transformations/hello/test
```

```
{
  "status": "ok",
  "output": {
    "greeting": "Hello, World!",
    "timestamp": "2026-05-05T14:30:00Z"
  },
```

```
  "duration_ms": 2
}
```

The test endpoint executes the transformation but does not count the call in Prometheus metrics. It is a safe way to verify before going live.

2.8 Send a Real Message

Now send a message through the data plane:

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"name": "Azure"}' \
  http://<ingress-url>:8085/api/transform/hello
```

```
{
  "greeting": "Hello, Azure!",
  "timestamp": "2026-05-05T14:30:05Z"
}
```

That is it. The transformation was compiled once at upload time. Every subsequent message executes the compiled version — typically one to five milliseconds per message.

2.9 What Just Happened

The following sequence describes the complete flow:

1. You uploaded `hello.utlx` to the Admin API on port 8081.
2. UTLXe parsed and compiled the transformation into an optimized in-memory representation.
3. The compiled transformation was registered in the engine and written to `/utlxe/data/transformations/hello/`.
4. The health endpoint switched to `ready: true`.
5. Your client sent a JSON message to the data plane on port 8085.
6. UTLXe looked up the compiled `hello` transformation, executed it, and returned the result.

The Admin API and the data plane run simultaneously — there is no "deployment mode" or downtime window.

2.10 Next Steps

- **Chapter 2** explains how to write more complex transformations — conditionals, array operations, multi-format output.
- **Chapter 3** covers the full Admin API — bundles, schemas, testing, and operational management.
- **Chapter 4** shows how to connect UTLXe to Azure Service Bus and Event Hub via Dapr.

3 Writing Transformations

This chapter teaches enough UTL-X to write production transformations. It is not the complete language reference — for that, see the companion book *UTL-X: One Language, All Formats*. Here you learn the patterns that cover ninety percent of real-world integration scenarios.

3.1 The .utlx File Structure

Every transformation has two parts: a header that declares the input and output formats, and a body that defines the transformation logic. They are separated by `---`.

```
%utlx 1.0
input json
output json
---
{
  orderId: $input.id,
  customer: $input.buyer.name,
  total: $input.amount
}
```

The header tells UTLXe how to parse the incoming message and how to serialize the output. The body is an expression that produces the output from the input.

3.2 Property Access

Access input fields with dot notation. The special variable `$input` refers to the incoming message.

```
$input.customer.name          // nested property
$input.items[0].price         // array index (0-based)
$input.@id                    // XML attribute
$input.order.item[2].@qty     // attribute on indexed element
```

If a property does not exist, the result is `null` — no error is thrown.

3.3 Object Construction

Curly braces create new objects. Each line is a key-value pair.

```
{
  orderId: $input.id,
  customer: $input.buyer.name,
  total: $input.quantity * $input.unitPrice
}
```

The spread operator copies all properties from an existing object:

```
{
  ...$input,
  processed: true,
  processedAt: now()
}
```

This produces the original input with two additional fields.

3.4 Let Bindings

Use `let` to name intermediate values. This avoids repetition and makes transformations readable.

```
{
  let subtotal = reduce($input.items, 0, (acc, i) -> acc + i.price)
  let tax = subtotal * 0.21
  let shipping = if (subtotal > 100) 0 else 9.95

  subtotal: subtotal,
  tax: round(tax, 2),
  shipping: shipping,
  total: round(subtotal + tax + shipping, 2)
}
```

3.5 Conditionals

The `if` expression chooses between two values:

```
if ($input.total > 1000) "premium" else "standard"
```

For multiple branches, use `match`:

```
match $input.status {
  "A" -> "Active",
  "I" -> "Inactive",
  "S" -> "Suspended",
  _ -> "Unknown"
}
```

The underscore `_` is the catch-all pattern.

3.6 Array Operations

Arrays are transformed with higher-order functions. The three most common are `map`, `filter`, and `reduce`.

map — transform each element:

```
map($input.items, (item) -> {
  name: item.product,
  total: item.quantity * item.unitPrice
})
```

filter — keep elements matching a condition:

```
filter($input.items, (item) -> item.price > 100)
```

reduce — aggregate to a single value:

```
reduce($input.items, 0, (sum, item) -> sum + item.price)
```

These compose naturally:

```
$input.orders
  | filter(_, (o) -> o.status == "active")
  | map(_, (o) -> {id: o.id, total: o.amount})
  | sortBy(_, (o) -> o.total)
```

The pipe operator `|` passes the result of each step to the next. The underscore `_` is a placeholder for the piped value.

3.7 Common Standard Library Functions

UTL-X has over 650 standard library functions. The ones you will use most often:

3.7.1 Strings

`concat(a, b, c)`	Concatenate strings
`upperCase(s)` / `lowerCase(s)`	Case conversion
`trim(s)`	Remove leading and trailing whitespace
`replace(s, old, new)`	Replace substring
`contains(s, sub)`	Check if string contains substring
`substring(s, start, end)`	Extract substring
`length(s)`	String length
`split(s, delimiter)`	Split string into array
`startsWith(s, prefix)`	Check prefix

3.7.2 Arrays

`map(arr, fn)`	Transform each element
`filter(arr, fn)`	Keep matching elements
`reduce(arr, init, fn)`	Aggregate to single value
`find(arr, fn)`	First matching element
`sortBy(arr, fn)`	Sort by key
`groupBy(arr, fn)`	Group by key
`flatMap(arr, fn)`	Map and flatten
`size(arr)`	Array length
`first(arr)` / `last(arr)`	First or last element

3.7.3 Math and Type

`round(n, decimals)`	Round to decimal places
`floor(n)` / `ceil(n)`	Floor or ceiling
`abs(n)` / `min(a, b)` / `max(a, b)`	Absolute value, min, max
`toString(v)` / `toNumber(v)`	Type conversion
`isNull(v)`	Null check
`now()`	Current timestamp
`formatDate(d, pattern)`	Format a date

3.8 Multi-Format Transformations

The header declares formats. The body works the same regardless of whether the input is JSON, XML, CSV, or YAML — UTL-X operates on the Universal Data Model (UDM), which is format-agnostic.

JSON to XML:

```
%utlx 1.0
input json
output xml
---
{
  Invoice: {
    ID: $input.orderId,
    Amount: $input.total,
    Currency: $input.currency
  }
}
```

The XML serializer produces `<Invoice><ID>12345</ID><Amount>250.0</Amount>...</Invoice>` from this object. You write objects — the format system handles serialization.

XML to JSON:

```
%utlx 1.0
input xml
output json
---
{
  orderId: $input.Invoice.ID,
  total: toNumber($input.Invoice.Amount),
  currency: $input.Invoice.Currency
}
```

CSV to JSON:

```
%utlx 1.0
input csv {header: true}
output json
---
map($input, (row) -> {
  name: row.Name,
  email: lowerCase(row.Email),
  active: row.Status == "A"
})
```

3.9 User-Defined Functions

Define reusable logic with `function`:

```
function discount(amount, tier) {
  match tier {
    "gold" -> amount * 0.10,
    "silver" -> amount * 0.05,
    _ -> 0
  }
}

{
  let disc = discount($input.total, $input.customerTier)
  orderId: $input.id,
  subtotal: $input.total,
```

```
  discount: round(disc, 2),
  total: round($input.total - disc, 2)
}
```

Functions are defined before the main expression. They can call other functions and use all standard library functions.

3.10 Worked Example: Invoice Transformation

A complete transformation that converts a Dynamics 365 Business Central order into a simplified UBL-like invoice:

```
%utlx 1.0
input json
output xml
---
{
  let order = $input
  let subtotal = reduce(order.lines, 0,
    (sum, line) -> sum + line.quantity * line.unitPrice)
  let tax = subtotal * 0.21

  Invoice: {
    ID: concat("INV-", order.orderNumber),
    IssueDate: formatDate(now(), "yyyy-MM-dd"),
    CustomerName: order.customer.name,
    Currency: order.currency,
    InvoiceLines: {
      InvoiceLine: map(order.lines, (line) -> {
        ID: line.lineNumber,
        Quantity: line.quantity,
        UnitPrice: line.unitPrice,
        LineTotal: round(line.quantity * line.unitPrice, 2)
      })
    },
    Subtotal: round(subtotal, 2),
    Tax: round(tax, 2),
    Total: round(subtotal + tax, 2)
  }
}
```

The XML serializer produces the UBL structure from this object. You write objects with property names that become element names — the format system handles the rest.

This transformation demonstrates property access, array iteration with `map`, aggregation with `reduce`, string concatenation, date formatting, and arithmetic.

3.11 Where to Learn More

- The complete language specification, including pattern matching, recursive functions, and the full operator table, is in the companion book *UTL-X: One Language, All Formats.*
- The standard library reference (all 650+ functions with examples) is in chapters 16 and 17 of the companion book.
- Example transformations are available in the project repository under `examples/`.

4 The Admin API

The Admin API is the primary interface for deploying and operating UTLXe on Azure. It runs on port 8081 alongside the health and metrics endpoints. This chapter covers every operation you need for day-to-day management.

4.1 Architecture: Two Ports, Two Purposes

UTLXe exposes two HTTP servers on separate ports:

Port	Purpose	Access
8081	Admin API + health + metrics	Internal to VNet — not exposed via ingress
8085	Data plane (message processing)	Exposed via Container App ingress

This separation allows network isolation. The data plane is accessible to client applications and Dapr sidecars. The admin port is restricted to operators and CI/CD pipelines within the VNet.

Both ports are always active. There is no mode switch — you can upload transformations while the data plane processes messages. See the architecture decision in the design document for the rationale.

4.2 Authentication

Every request to `/admin/*` endpoints must include the `X-Admin-Key` header:

```
curl -H "X-Admin-Key: my-secret-key-here" \
  http://<internal-ip>:8081/admin/transformations
```

The key is set via the `UTLXE_ADMIN_KEY` environment variable. If this variable is not set, all admin endpoints return 403 — the API is locked by default.

The health endpoints (`/health`, `/metrics`) do not require authentication. They are read-only and needed by Kubernetes probes and Prometheus.

4.3 Uploading Transformations

4.3.1 Single .utlx File

The simplest deployment: upload just the `.utlx` source file. UTLXe applies sensible defaults (COMPILED strategy, auto-detect format).

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl.utlx" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

Response:

```
{
  "status": "deployed",
  "name": "invoice-to-ubl",
  "strategy": "COMPILED",
  "config": "defaults",
  "compiled_in_ms": 48
}
```

4.3.2 With Configuration

For explicit control over strategy and validation, include a `transform.yaml`:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl.utlx" \
  -F "config=@transform.yaml" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

A typical `transform.yaml`:

```
strategy: COMPILED
validationPolicy: strict
inputs:
  - name: input
    schema: order.json
maxConcurrent: 4
```

4.3.3 ZIP Bundle

For batch deployment, upload a ZIP containing all transformations and schemas at once. This replaces the entire bundle atomically.

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "file=@bundle.zip" \
  http://<admin>:8081/admin/bundle
```

The ZIP follows this structure:

```
bundle.zip
  schemas/
    order.xsd
    invoice.json
  transformations/
    invoice-to-ubl/
      invoice-to-ubl.utlx
    order-enrichment/
      order-enrichment.utlx
```

The `schemas/` directory is optional. Each transformation directory requires only the `.utlx` source file. A `transform.yaml` can be added alongside the `.utlx` to override defaults (strategy, validation policy, output binding) — but it is not required.

4.4 Managing Schemas

Schemas are shared resources, stored separately from transformations. A single schema like `order.xsd` can be referenced by multiple transformations.

Upload a schema:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "file=@order.xsd" \
  http://<admin>:8081/admin/schemas/order.xsd
```

List all schemas:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/schemas
```

```
{
  "schemas": [
    {"filename": "order.xsd", "size_bytes": 4820, "uploaded_at": "2026-05-05T14:29:00Z"},
    {"filename": "invoice.json", "size_bytes": 2340, "uploaded_at":
"2026-05-05T14:29:05Z"}
  ]
}
```

Updating a schema does not recompile transformations — schemas are resolved at validation time, not compile time. The new schema takes effect on the next message.

4.5 Testing Before Go-Live

The test endpoint is the most important step after uploading a transformation. It executes the transformation with sample input and returns the result without affecting metrics.

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"orderId": "12345", "amount": 250.00, "currency": "EUR"}' \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/test
```

On success:

```
{
  "status": "ok",
  "output": {"Invoice": {"ID": "INV-12345", "Amount": 250.0, "Currency": "EUR"}},
  "duration_ms": 3
}
```

On failure:

```
{
  "status": "error",
  "error": "Null reference: $input.customer.address",
  "line": 14,
  "column": 22
}
```

Always test before routing real traffic. The deployment workflow should be: upload, test, then allow traffic.

4.6 Listing and Inspecting

List all deployed transformations:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations
```

```
{
  "transformations": [
    {
      "name": "invoice-to-ubl",
      "strategy": "COMPILED",
      "status": "ready",
      "config": "explicit",
      "deployed_at": "2026-05-05T14:30:00Z",
      "messages_processed": 12345,
      "avg_transform_ms": 2.3
    }
  ]
}
```

Get details for a specific transformation, including the source code:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

4.7 Updating Transformations

Upload the same name again to replace a transformation. The update is atomic:

1. The new source is compiled.
2. If compilation fails, the upload is rejected — the running transformation is untouched.
3. If compilation succeeds, the new version replaces the old one in the registry.
4. In-flight messages on the old version complete normally.
5. New messages use the new version immediately.

There is no downtime, no mode switch, and no restart.

4.8 Deleting

Remove a single transformation:

```
curl -X DELETE \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

Remove everything and start fresh:

```
curl -X DELETE \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle
```

4.9 Exporting the Bundle

Download the current state as a ZIP file:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle -o bundle-backup.zip
```

This is useful for version control, backup, or migrating transformations to another container. The exported ZIP has the same format as the upload — you can re-import it with `POST /admin/bundle`.

4.10 Data Plane Discovery

Client applications can discover available transformations without an admin key. This endpoint runs on the data plane port (8085):

```
curl http://<ingress>:8085/api/transformations
```

```
{
  "transformations": [
    {"name": "invoice-to-ubl", "status": "ready", "input": "json", "output": "xml"},
    {"name": "order-enrichment", "status": "ready", "input": "json", "output": "json"}
  ]
}
```

This makes the data plane self-describing. A client connecting for the first time can discover what transformations are available without out-of-band knowledge.

4.11 Engine Info

Get basic operational information:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/info
```

```
{
  "version": "1.0.1",
  "uptime_seconds": 86400,
  "mode": "http",
  "workers": 4,
  "heap_max_mb": 1536,
  "data_dir": "/utlxe/data",
  "persistence": "volume-backed",
  "admin_key_set": true,
  "transformations": 3,
  "schemas": 2,
  "ready": true
}
```

4.12 Two Deployment Workflows

The Admin API supports two workflows. Use whichever fits your process.

Batch workflow — assemble a ZIP in your CI/CD pipeline and deploy in one call:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@bundle.zip" \
  http://<admin>:8081/admin/bundle
```

Incremental workflow — upload resources one at a time:

```
# Upload schemas first
curl -X POST -H "X-Admin-Key: $KEY" -F "file=@order.xsd" .../admin/schemas/order.xsd

# Then transformations
```

```
curl -X POST -H "X-Admin-Key: $KEY" -F "source=@invoice.utlx" .../admin/transformations/
invoice

# Export the assembled bundle for version control
curl -H "X-Admin-Key: $KEY" .../admin/bundle -o my-bundle.zip
```

Both workflows produce the same result. You can start with the incremental workflow during development, then switch to batch for production CI/CD.

Part II

Integration

Connect to Azure services and monitor your deployment

5 Connecting to Azure Services

This chapter walks through every step of connecting UTLXe to Azure Service Bus and Event Hub. Each section is a complete, self-contained walkthrough — from creating the Azure resources to sending the first message through UTLXe.

5.1 Prerequisites

Before you begin, you need:

- An Azure subscription.
- The Azure CLI installed (`az` command). Install from `https://docs.microsoft.com/en-us/cli/azure/install-azure-cli`.
- A deployed UTLXe Container App (see Chapter 2: Quick Start).
- The UTLXe admin key (`UTLXE_ADMIN_KEY`) configured on the container.

All commands in this chapter use the Azure CLI. Where relevant, the equivalent Azure Portal steps are noted.

5.2 Walkthrough: Service Bus Queue to Queue

This walkthrough connects an Azure Service Bus input queue to UTLXe, transforms messages, and sends results to an output queue.

5.2.1 Step 1: Create the Service Bus Namespace and Queues

```
# Variables — adjust to your environment
RESOURCE_GROUP="myResourceGroup"
LOCATION="westeurope"
SB_NAMESPACE="sb-utlxe-demo"

# Create the Service Bus namespace
az servicebus namespace create \
  --name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku Standard

# Create the input queue
az servicebus queue create \
  --name incoming-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP

# Create the output queue
az servicebus queue create \
  --name processed-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP
```

Portal alternative: In the Azure Portal, go to "Create a resource" > "Service Bus" > fill in namespace name, resource group, region, and Standard tier. Then open the namespace and click "+ Queue" twice to create `incoming-orders` and `processed-orders`.

5.2.2 Step 2: Get the Connection String

```
# Get the connection string for the Dapr component
az servicebus namespace authorization-rule keys list \
  --name RootManageSharedAccessKey \
```

```
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --query primaryConnectionString -o tsv
```

Copy the connection string. You will store it as a secret in the Container App environment.

5.2.3 Step 3: Store the Connection String as a Secret

```
CONTAINER_APP="utlxe"
CONTAINER_ENV="my-container-env"

# Store as a secret in the Container App
az containerapp secret set \
  --name $CONTAINER_APP \
  --resource-group $RESOURCE_GROUP \
  --secrets servicebus-connection="<paste connection string>"
```

Portal alternative: Open the Container App > "Secrets" > "+ Add" > name: `servicebus-connection`, value: paste the connection string.

5.2.4 Step 4: Configure the Dapr Input Component

Enable Dapr on the Container App and add the input binding component:

```
# Enable Dapr on the Container App (if not already enabled)
az containerapp dapr enable \
  --name $CONTAINER_APP \
  --resource-group $RESOURCE_GROUP \
  --dapr-app-id utlxe \
  --dapr-app-port 8085

# Create the Dapr component for the input queue
az containerapp env dapr-component set \
  --name $CONTAINER_ENV \
  --resource-group $RESOURCE_GROUP \
  --dapr-component-name orders-in \
  --yaml dapr-input.yaml
```

Create `dapr-input.yaml`:

```
componentType: bindings.azure.servicebusqueues
version: v1
metadata:
  - name: connectionString
    secretRef: servicebus-connection
  - name: queueName
    value: "incoming-orders"
scopes:
  - utlxe
```

Portal alternative: Open the Container App Environment > "Dapr components" > "+ Add" > component name: `orders-in`, type: `bindings.azure.servicebusqueues`. Add metadata: `connectionString` (secret reference: `servicebus-connection`), `queueName`: `incoming-orders`. Scope to `utlxe`.

5.2.5 Step 5: Configure the Dapr Output Component

```
az containerapp env dapr-component set \
  --name $CONTAINER_ENV \
  --resource-group $RESOURCE_GROUP \
  --dapr-component-name orders-out \
  --yaml dapr-output.yaml
```

Create `dapr-output.yaml`:

```
componentType: bindings.azure.servicebusqueues
version: v1
metadata:
  - name: connectionString
    secretRef: servicebus-connection
  - name: queueName
    value: "processed-orders"
scopes:
  - utlxe
```

5.2.6 Step 6: Upload the Transformation

Create `orders-in.utlx` — the transformation name must match the Dapr input component name (`orders-in`):

```
%utlx 1.0
input json
output json
---
{
  processedOrderId: concat("PROC-", $input.orderId),
  customer: upperCase($input.customerName),
  total: round($input.quantity * $input.unitPrice, 2),
  processedAt: now()
}
```

Upload it with the output binding configured:

```
# Upload the transformation
curl -X POST \
  -H "X-Admin-Key: $ADMIN_KEY" \
  -F "source=@orders-in.utlx" \
  http://<admin-ip>:8081/admin/transformations/orders-in

# Optionally set the output binding via config
curl -X POST \
  -H "X-Admin-Key: $ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"outputBinding": "orders-out"}' \
  http://<admin-ip>:8081/admin/transformations/orders-in/config
```

5.2.7 Step 7: Test the Transformation

Before sending real messages, test with sample input:

```
curl -X POST \
  -H "X-Admin-Key: $ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"orderId":"ORD-001","customerName":"Acme Corp","quantity":5,"unitPrice":24.50}' \
  http://<admin-ip>:8081/admin/transformations/orders-in/test
```

Expected response:

```
{
  "status": "ok",
  "output": {
    "processedOrderId": "PROC-ORD-001",
    "customer": "ACME CORP",
    "total": 122.5,
    "processedAt": "2026-05-05T14:30:00Z"
  },
  "duration_ms": 2
}
```

5.2.8 Step 8: Send a Message to Service Bus

```
# Send a test message to the input queue
az servicebus queue send \
  --name incoming-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --body '{"orderId":"ORD-001","customerName":"Acme Corp","quantity":5,"unitPrice":24.50}'
```

Portal alternative: Open the Service Bus namespace > "incoming-orders" queue > "Service Bus Explorer" > "Send message" > paste the JSON body > click "Send".

5.2.9 Step 9: Check the Output Queue

```
# Peek at the output queue
az servicebus queue peek \
  --name processed-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP
```

You should see the transformed message:

```
{
  "processedOrderId": "PROC-ORD-001",
  "customer": "ACME CORP",
  "total": 122.5,
  "processedAt": "2026-05-05T14:30:05Z"
}
```

The complete flow is now working: Service Bus input queue → Dapr → UTLXe → Dapr → Service Bus output queue.

5.2.10 What Happens on Failure

If the transformation fails (for example, a required field is missing), UTLXe returns HTTP 500 to Dapr. Dapr does not acknowledge the message on Service Bus. Service Bus retries delivery based on the queue's retry policy. After the maximum number of retries, the message moves to the dead-letter queue.

Check recent errors:

```
curl -H "X-Admin-Key: $ADMIN_KEY" \
  http://<admin-ip>:8081/admin/transformations/orders-in/errors
```

5.3 Walkthrough: Event Hub

Event Hub uses the same pattern with a different Dapr component type. Only the differences from the Service Bus walkthrough are shown.

5.3.1 Create the Event Hub

```
EH_NAMESPACE="eh-utlxe-demo"

az eventhubs namespace create \
  --name $EH_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku Standard

az eventhubs eventhub create \
  --name incoming-events \
  --namespace-name $EH_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --partition-count 4

# Storage account for checkpointing
az storage account create \
  --name stutlxecheckpoint \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku Standard_LRS

az storage container create \
  --name checkpoints \
  --account-name stutlxecheckpoint
```

5.3.2 Dapr Component

Create `dapr-eventhub-input.yaml`:

```
componentType: bindings.azure.eventhubs
version: v1
metadata:
  - name: connectionString
    secretRef: eventhub-connection
  - name: consumerGroup
    value: "utlxe-consumer"
  - name: storageAccountName
    value: "stutlxecheckpoint"
  - name: storageContainerName
    value: "checkpoints"
```

```
  - name: storageAccountKey
    secretRef: storage-account-key
scopes:
  - utlxe
```

The transformation is identical to the Service Bus example — UTLXe does not know or care whether the message came from Service Bus or Event Hub.

5.4 Direct HTTP (No Dapr)

For synchronous request/response patterns, clients call UTLXe directly on port 8085. No Dapr configuration is needed.

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"orderId":"ORD-001","customerName":"Acme Corp","quantity":5,"unitPrice":24.50}' \
  http://<ingress-url>:8085/api/transform/orders-in
```

The response is the transformed message, returned synchronously. This is suitable for:

- API gateway integration — transform requests or responses inline.
- Batch processing — send messages from a script or pipeline.
- Testing and development.

No Dapr component, no Service Bus, no queue configuration. Just HTTP in, HTTP out.

5.5 Worked Example: Dynamics 365 to Peppol

A complete end-to-end flow for European e-invoicing:

1. **Dynamics 365 Business Central** publishes an invoice as OData JSON to a Service Bus queue (using D365's built-in Service Bus integration).
2. **Dapr** delivers the message to UTLXe.
3. **UTLXe** runs the `invoice-to-ubl` transformation — converting D365 JSON to UBL 2.1 XML with Peppol BIS 3.0 compliance.
4. **Dapr** publishes the UBL XML to an output Service Bus topic.
5. A **Peppol Access Point** subscribes to the topic and delivers the invoice via AS4.

The transformation, Dapr components, and Service Bus resources are configured following the same steps shown in this chapter. The only difference is the `.utlx` content — which maps D365 fields to UBL elements.

5.6 Summary: What You Configure Where

What	Where	How
Service Bus / Event Hub	Azure Portal or CLI	`az servicebus` / `az eventhubs` commands
Connection string	Container App secrets	`az containerapp secret set`
Dapr input binding	Container App Environment	Dapr component YAML via CLI or Portal
Dapr output binding	Container App Environment	Dapr component YAML via CLI or Portal
Transformation	UTLXe Admin API	`POST /admin/transformations/{name}`
Output binding reference	UTLXe Admin API	`POST /admin/transformations/{name}/config`

The Azure resources and Dapr components are configured once. The transformations can be updated at any time without changing the infrastructure.

6 Monitoring and Observability

Production deployments need visibility into what UTLXe is doing. This chapter covers health probes, Prometheus metrics, and how to set up monitoring dashboards.

6.1 Health Endpoint

The health endpoint runs on port 8081 and serves two purposes: Kubernetes probes and operational status checks.

```
curl http://<internal-ip>:8081/health
```

```
{
  "status": "UP",
  "transformations": 3,
  "ready": true
}
```

The fields:

`status`	`"UP"` if the process is alive. Used by the liveness probe.
`transformations`	Number of compiled transformations in the registry.
`ready`	`true` when at least one transformation is loaded and compiled. Used by the readiness probe.

6.1.1 Liveness vs. Readiness

Probe	Checks	Action if fails
Liveness	`status == "UP"`	Kubernetes restarts the container
Readiness	`ready == true`	Kubernetes stops routing traffic to this instance

A container that just started but has not received its bundle yet is **alive but not ready**. Traffic is held until transformations are uploaded and compiled. This prevents errors during the deployment window.

6.2 Prometheus Metrics

UTLXe exposes Prometheus metrics on the same port:

```
curl http://<internal-ip>:8081/metrics
```

6.2.1 Per-Transformation Metrics

```
utlxe_messages_total{transform="invoice-to-ubl"} 12345
utlxe_transform_duration_seconds{transform="invoice-to-ubl",quantile="0.50"} 0.002
utlxe_transform_duration_seconds{transform="invoice-to-ubl",quantile="0.95"} 0.005
utlxe_transform_duration_seconds{transform="invoice-to-ubl",quantile="0.99"} 0.012
utlxe_errors_total{transform="invoice-to-ubl"} 3
```

6.2.2 Engine-Level Metrics

```
utlxe_active_threads 4
utlxe_heap_used_bytes 524288000
utlxe_uptime_seconds 86400
```

These metrics are standard Prometheus format and can be scraped by any Prometheus-compatible system.

6.3 Setting Up Azure Monitor

Azure Container Apps integrates with Azure Monitor out of the box. Container logs are available in the Log Analytics workspace associated with the Container App Environment.

To view container logs:

```
az containerapp logs show \
  -n utlxe -g myResourceGroup \
  --follow
```

UTLXe logs in structured format: timestamp, level, transformation name, and message. Filter by transformation name to isolate issues.

For Prometheus metrics, configure Azure Monitor managed Prometheus or deploy a Prometheus instance that scrapes port 8081.

6.4 Grafana Dashboard

A recommended Grafana dashboard layout for UTLXe:

Row 1 — Throughput:

- Messages per second (rate of `utlxe_messages_total`)
- Errors per second (rate of `utlxe_errors_total`)
- Error rate percentage

Row 2 — Latency:

- p50 latency (median transformation time)
- p95 latency (tail latency)
- p99 latency (worst case)

Row 3 — Resources:

- Heap usage (percentage of max)
- Active threads
- Container CPU and memory from Azure Monitor

6.4.1 Alert Rules

Configure alerts for:

Metric	Threshold	Action
Error rate	> 1% for 5 minutes	Notify on-call
p99 latency	> 500ms for 5 minutes	Investigate — possible GC pressure or large messages
Heap usage	> 80% of max	Warning — consider upgrading plan
Transformations	**6.5 0 for 2 minutes**	Container restarted without bundle — check persistence

6.6 Error Diagnosis

Prometheus gives you counters, but not the actual error messages. For quick diagnosis, use the error ring buffer:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/errors?limit=10
```

```
{
  "errors": [
    {
      "timestamp": "2026-05-05T14:32:01Z",
      "message": "Null reference: $input.customer.address",
      "line": 14,
      "input_preview": "{\"orderId\":\"12345\",\"customer\":{\"name\":\"Acme\"}}"
    }
  ],
  "total_errors": 47,
  "showing": 10
}
```

The `input_preview` is truncated to 200 characters to avoid exposing full message payloads. This is usually enough to identify the pattern that causes failures.

The typical diagnosis workflow:

1. Notice elevated error rate in Grafana.
2. Check the error ring buffer for the affected transformation.
3. Identify the pattern (missing field, unexpected type, schema mismatch).
4. Pause the transformation if needed (Chapter 6).
5. Fix and re-upload.
6. Test with sample input.
7. Resume.

Part III

Production

Operations, scaling, security, and automation

7 Operations

This chapter covers day-to-day operational tasks: updating transformations without downtime, handling incidents, and managing validation at runtime.

7.1 Updating a Transformation

Upload a new version of the same transformation name. The update is atomic and zero-downtime:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl-v2.utlx" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

What happens internally:

1. UTLXe compiles the new source. If compilation fails, the upload is rejected and the running version is untouched.
2. The new compiled version atomically replaces the old one in the registry.
3. Messages that are currently being processed on the old version complete normally — they hold a reference to the old compiled code.
4. New messages arriving after the swap use the new version.

There is no restart, no mode switch, and no window where messages are dropped.

7.2 Pause and Resume

During an incident, you may need to stop processing for a specific transformation without removing it. Pausing preserves the compiled transformation and its on-disk state while stopping traffic.

Pause:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/pause
```

While paused:

- The data plane returns 503 for that transformation. Other transformations continue normally.
- Dapr receives the 503 and does not acknowledge the message. Service Bus retries it later (or moves it to the dead-letter queue after max retries).
- The transformation stays compiled in memory and on disk — resume is instant.
- The listing shows `"status": "paused"`.

Resume:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/resume
```

Processing resumes immediately. Queued messages on Service Bus are delivered again.

7.3 Incident Response Workflow

A complete incident lifecycle:

1. **Detect** — Grafana alert fires on elevated error rate.
2. **Diagnose** — Check the error ring buffer:

```
curl -H "X-Admin-Key: $KEY" \
  .../admin/transformations/invoice-to-ubl/errors
```

3. **Contain** — Pause the transformation:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  .../admin/transformations/invoice-to-ubl/pause
```

4. **Fix** — Edit the `.utlx` source to handle the failing case.
5. **Deploy** — Upload the fix:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl-fixed.utlx" \
  .../admin/transformations/invoice-to-ubl
```

6. **Verify** — Test with the input that caused the failure:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"orderId":"12345","customer":null}' \
  .../admin/transformations/invoice-to-ubl/test
```

7. **Resume** — Bring the transformation back online:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  .../admin/transformations/invoice-to-ubl/resume
```

The entire cycle — from detection to resume — can happen in minutes without a container restart.

7.4 Validation Override

Schema validation catches malformed messages before they reach the transformation logic. But during an incident, validation might be too strict — blocking messages that are technically valid but don't match the expected schema exactly.

Set a runtime override:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"policy": "off"}' \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/validation
```

Check the effective state:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/validation
```

```
{
  "effective_policy": "off",
  "source": "runtime-override",
  "config_policy": "strict",
  "header_policy": "strict"
}
```

Remove the override to revert to the configured policy:

```
curl -X DELETE \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/validation
```

The override is ephemeral — it is not written to disk and disappears on container restart. The precedence chain is:

Level	Set by	Persists
Runtime override	Ops via Admin API	No — ephemeral
`transform.yaml` config	Ops at deploy time	Yes — on disk
`.utlx` header	Developer at dev time	Yes — in source
Default	—	`strict`

The highest-priority level that is set wins.

7.5 Updating Schemas

Replace a schema without recompiling transformations:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "file=@order-v2.xsd" \
  http://<admin>:8081/admin/schemas/order.xsd
```

The new schema takes effect on the next message. Transformations that reference `order.xsd` do not need to be re-uploaded — schema resolution happens at validation time, not compile time.

7.6 Bulk Operations

Replace everything at once (atomic):

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@new-bundle.zip" \
  http://<admin>:8081/admin/bundle
```

Remove everything and start fresh:

```
curl -X DELETE -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle
```

Export the current state as a backup:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle -o backup.zip
```

7.7 Engine Configuration

View the current engine configuration:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/config
```

Update a runtime-safe field:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"maxInputSize": "10MB"}' \
  http://<admin>:8081/admin/config
```

Fields that require a restart (like port numbers) are accepted but flagged in the response:

```
{"updated": ["maxInputSize"], "restart_required": []}
```

8 Persistence and Scaling

This chapter explains how transformations survive container restarts and how to scale UTLXe for production throughput.

8.1 The Persistence Problem

Docker containers have an ephemeral filesystem. When a container restarts — due to a crash, a scale-to-zero event, or a redeployment — everything written inside the container is lost. This means the `/utlxe/data/` directory, where uploaded transformations and schemas are stored, is wiped clean.

This is a fundamental container property, not something UTLXe can fix internally. The solution is external storage.

8.2 Azure Files Volume Mount

The recommended approach for the Azure Marketplace is an Azure File Share mounted at `/utlxe/data/`. The mount is configured by the platform at the infrastructure level — before the container starts. UTLXe uses standard Java file I/O to read and write the directory. It has no knowledge that the directory is backed by a network file share.

```
Container filesystem (ephemeral):
  /utlxe/utlxe.jar             from Docker image, rebuilt on restart

Mount point (persistent):
  /utlxe/data/                  Azure File Share — survives restarts
    schemas/
      order.xsd
    transformations/
      invoice-to-ubl/
        invoice-to-ubl.utlx
      order-enrichment/
        order-enrichment.utlx
```

On restart, UTLXe scans `/utlxe/data/`, finds the transformations from the previous session, compiles them, and becomes ready. Zero manual intervention.

To enable persistent storage, check the "Enable persistent transformation storage" option during Marketplace deployment. This creates an Azure Storage Account and File Share, and configures the volume mount in the Container App.

8.3 CI/CD Re-Deploy Pattern

An alternative to persistent storage: let the CI/CD pipeline re-deploy transformations after every container start.

1. Container starts with an empty `/utlxe/data/`.
2. Health endpoint returns `ready: false`.
3. The CI/CD pipeline detects the not-ready state and uploads the bundle.
4. UTLXe compiles the transformations.
5. Health endpoint switches to `ready: true`.
6. Kubernetes routes traffic.

The bundle lives in the CI/CD system (a git repository or artifact store). The container is truly stateless — the pipeline is the source of truth.

8.4 Startup Sequence

Regardless of persistence mode, the startup sequence is:

1. Start the HTTP server on port 8081 (health, metrics, admin API).
2. Scan `/utlxe/data/` for existing transformations and schemas.
   - If found (volume mount from previous session): compile and register them.
   - If empty: wait for API uploads.
3. Start the data plane on port 8085.
4. The readiness probe checks `ready == true` before Kubernetes routes traffic.

The admin API is available from step 1 — you can upload transformations while the startup scan is still running.

8.5 Memory Sizing

UTLXe runs on the JVM with a configured heap size. The heap must be large enough to hold the message being processed plus the intermediate objects created during transformation.

Plan	Container	Heap	Max message
Starter	2 GB	1536 MB	50 KB
Professional	4 GB	3072 MB	200 KB

The heap is set to 75% of the container memory. The remaining 25% is for the JVM itself (metaspace, thread stacks, native memory) and the operating system.

The `-XX:+AlwaysPreTouch` JVM flag allocates the entire heap at startup. If the container does not have enough RAM, the process fails immediately with a clear error — rather than crashing hours later under load.

8.6 Horizontal Scaling

For higher throughput, deploy multiple container replicas behind the Azure load balancer. Each replica runs an independent UTLXe instance.

If using persistent storage, all replicas share the same Azure File Share. Upload transformations once — all replicas see the same files.

If using CI/CD re-deploy, the pipeline must deploy to each replica (or use a shared storage backend for the bundle).

Azure Service Bus distributes messages across consumers automatically. Event Hub uses consumer groups to distribute partitions across replicas.

8.7 Never Use Swap

Swap and garbage collection do not mix. When the JVM's garbage collector scans the heap, it touches every page. If those pages have been swapped to disk, each page access triggers a page fault — a disk I/O operation that takes milliseconds instead of nanoseconds. A GC pause that normally takes 20 milliseconds can take 20 *seconds* when pages are swapped.

The rule: **never run UTLXe with swap enabled.**

In Azure Container Apps, set the memory request equal to the memory limit:

```
resources:
  requests:
    memory: "2Gi"
  limits:
    memory: "2Gi"
```

This ensures the container gets exactly the memory it requests, with no swap. If the container exceeds its memory limit, Kubernetes kills it (OOM) — which is faster to recover from than swap thrashing.

8.8 Poison Messages

A poison message is an input that is too large or too malformed to process. Without protection, a single 2 GB message can exhaust the JVM heap and crash the container.

The `maxInputSize` configuration rejects oversized messages before parsing:

```
maxInputSize: 5MB
```

Messages larger than this limit are rejected immediately with a 413 status code. The Dapr sidecar receives the error, and Service Bus moves the message to the dead-letter queue after max retries.

The default is 5 MB. Increase it only if you know your messages are legitimately larger, and verify that the heap has enough room.

9 Security

This chapter covers the security model of a UTLXe deployment: network isolation, authentication, secrets management, and the principle of least privilege.

9.1 Network Architecture

UTLXe uses port separation to isolate management traffic from data traffic:

Port	Scope	Contains
8085	Public (via ingress)	Data plane — message processing only
8081	Internal (VNet only)	Admin API, health probes, Prometheus metrics

The Container App ingress exposes only port 8085 to the outside world. Port 8081 is accessible only within the VNet, which means:

- Client applications can send messages for transformation but cannot upload, delete, or modify transformations.
- Operators and CI/CD pipelines access the admin API from within the VNet (or via VPN/private endpoint).
- Prometheus scrapes metrics from port 8081 inside the VNet.
- Kubernetes probes check health on port 8081 inside the VNet.

9.2 Admin API Authentication

The admin API is protected by an API key. Every request to `/admin/*` must include:

```
X-Admin-Key: <value of UTLXE_ADMIN_KEY>
```

If `UTLXE_ADMIN_KEY` is not set, all admin endpoints return 403. The API is locked by default — there is no open-by-default behavior.

Store the key as a Container App secret:

```
az containerapp secret set \
  -n utlxe -g myResourceGroup \
  --secrets admin-key=<your-secret-key>
```

Reference it in the environment:

```
az containerapp update \
  -n utlxe -g myResourceGroup \
  --set-env-vars UTLXE_ADMIN_KEY=secretref:admin-key
```

Container App secrets are encrypted at rest and injected as environment variables at runtime. They are not visible in logs or in the container filesystem.

To rotate the key, update the secret and restart the container. No code change is needed.

9.3 Non-Root Container

The UTLXe Docker image runs as a non-root user (`utlxe:utlxe`). The Dockerfile creates a dedicated user and group:

```
RUN groupadd -r utlxe && useradd -r -g utlxe utlxe
USER utlxe
```

The container has no elevated privileges. The only writable directory is `/utlxe/data/`, where the Admin API stores uploaded transformations and schemas.

9.4 Secrets in Transformations

Transformations sometimes need secrets — API keys, connection strings, tokens. Never hardcode secrets in `.utlx` files. Instead, use the `env()` function to read environment variables:

```
%utlx 1.0
input json
output json
---
{
  ...$input,
  authHeader: concat("Bearer ", env("API_TOKEN"))
}
```

The `API_TOKEN` value comes from a Container App secret, injected as an environment variable. The `.utlx` source can be committed to version control without exposing the secret.

The `env()` function is restricted in the CLI for security, but available in the engine (UTLXe) where environment variables are controlled by the deployment configuration.

9.5 VNet Integration

For production deployments, place the Container App Environment in a VNet:

- The admin port (8081) is accessible only from within the VNet.
- The data plane ingress (8085) can be configured as internal (VNet only) or external (public with TLS).
- Service Bus and Storage Account connections use private endpoints — no traffic over the public internet.

A typical production network layout:

UTLXe Container App	VNet, private subnet
Admin access	VNet only (or via Azure Bastion / VPN)
Data plane ingress	Internal or external, TLS terminated by Azure
Service Bus	Private endpoint in same VNet
Storage Account	Private endpoint in same VNet (for Azure Files)
Prometheus	In same VNet, scrapes port 8081

9.6 TLS and HTTPS

UTLXe itself runs HTTP on both ports. TLS termination is handled by the Container App ingress for port 8085. Internal traffic between Dapr, UTLXe, and other services within the VNet does not need TLS — it is already isolated by the network boundary.

If your security policy requires TLS on internal traffic, configure the Container App Environment with mTLS (mutual TLS), which encrypts all traffic between containers in the environment.

10 CI/CD Integration

Automated deployment ensures that transformations are tested, versioned, and deployed consistently. This chapter shows how to integrate UTLXe with GitHub Actions and Azure DevOps.

10.1 The Deployment Model

The transformation source lives in a git repository. The CI/CD pipeline:

1. Assembles a bundle (ZIP) from the repository contents.
2. Validates the bundle against the running UTLXe instance (dry run).
3. Deploys the bundle.
4. Verifies with test inputs.

No custom Docker image is built. The UTLXe container from the Marketplace stays as-is. Only the transformations change.

10.2 Repository Structure

A recommended layout for the transformation repository:

```
my-transformations/
  schemas/
    order.xsd
    invoice.json
  transformations/
    invoice-to-ubl/
      invoice-to-ubl.utlx
    order-enrichment/
      order-enrichment.utlx
  test-data/
    invoice-to-ubl/
      input.json
      expected-output.xml
  scripts/
    build-bundle.sh
    deploy.sh
```

The `test-data/` directory contains sample inputs and expected outputs for each transformation. The pipeline uses these for post-deployment verification.

10.3 Building the Bundle

A simple script assembles the ZIP:

```
#!/bin/bash
# build-bundle.sh
cd "$(dirname "$0")/.."
zip -r bundle.zip schemas/ transformations/
echo "Bundle created: bundle.zip"
```

10.4 GitHub Actions Workflow

```
name: Deploy Transformations
on:
  push:
    branches: [main]
```

```yaml
env:
  UTLXE_ADMIN_URL: ${{ secrets.UTLXE_ADMIN_URL }}
  UTLXE_ADMIN_KEY: ${{ secrets.UTLXE_ADMIN_KEY }}
  UTLXE_DATA_URL: ${{ secrets.UTLXE_DATA_URL }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build bundle
        run: |
          cd transformations
          zip -r ../bundle.zip schemas/ transformations/

      - name: Validate (dry run)
        run: |
          STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
            -X POST \
            -H "X-Admin-Key: $UTLXE_ADMIN_KEY" \
            -F "file=@bundle.zip" \
            "$UTLXE_ADMIN_URL/admin/bundle/validate")
          if [ "$STATUS" != "200" ]; then
            echo "Validation failed with status $STATUS"
            exit 1
          fi

      - name: Deploy
        run: |
          curl -sf \
            -X POST \
            -H "X-Admin-Key: $UTLXE_ADMIN_KEY" \
            -F "file=@bundle.zip" \
            "$UTLXE_ADMIN_URL/admin/bundle"

      - name: Verify
        run: |
          for dir in test-data/*/; do
            NAME=$(basename "$dir")
            INPUT="$dir/input.json"
            EXPECTED="$dir/expected-output.xml"
            if [ -f "$INPUT" ]; then
              RESULT=$(curl -sf \
                -X POST \
                -H "X-Admin-Key: $UTLXE_ADMIN_KEY" \
                -H "Content-Type: application/json" \
                -d @"$INPUT" \
                "$UTLXE_ADMIN_URL/admin/transformations/$NAME/test")
              echo "$NAME: $RESULT"
            fi
          done
```

The workflow validates before deploying, and verifies after. If validation fails, the deployment never happens. If verification fails, the pipeline reports the error — the transformations are already deployed, but the team is notified.

10.5 Azure DevOps Pipeline

The structure is similar, using Azure CLI tasks for authentication:

```yaml
trigger:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  - group: utlxe-secrets

steps:
  - script: |
      cd transformations
      zip -r $(Build.ArtifactStagingDirectory)/bundle.zip schemas/ transformations/
    displayName: 'Build bundle'

  - script: |
      curl -sf -X POST \
        -H "X-Admin-Key: $(UTLXE_ADMIN_KEY)" \
        -F "file=@$(Build.ArtifactStagingDirectory)/bundle.zip" \
        "$(UTLXE_ADMIN_URL)/admin/bundle/validate"
    displayName: 'Validate'

  - script: |
      curl -sf -X POST \
        -H "X-Admin-Key: $(UTLXE_ADMIN_KEY)" \
        -F "file=@$(Build.ArtifactStagingDirectory)/bundle.zip" \
        "$(UTLXE_ADMIN_URL)/admin/bundle"
    displayName: 'Deploy'
```

Store `UTLXE_ADMIN_KEY` and `UTLXE_ADMIN_URL` in an Azure DevOps variable group marked as secret.

10.6 Rollback

Rollback is straightforward: re-deploy a previous bundle. Keep previous bundles as pipeline artifacts or in a git tag.

```
# Download previous bundle from artifact storage
# Re-deploy it
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@bundle-v1.2.3.zip" \
  http://<admin>:8081/admin/bundle
```

The bundle upload is atomic — the old transformations are replaced in one operation. There is no partial state.

Alternatively, use the export endpoint to create a backup before deploying:

```
# Backup current state
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle -o backup-before-deploy.zip
```

```
# Deploy new version
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@new-bundle.zip" \
  http://<admin>:8081/admin/bundle

# If something goes wrong, rollback
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@backup-before-deploy.zip" \
  http://<admin>:8081/admin/bundle
```

10.7 Multi-Environment Promotion

Use the same bundle across environments with environment-specific configuration:

Environment	Validation policy	How
Development	`off`	Runtime override via Admin API
Staging	`warn`	`transform.yaml` config
Production	`strict`	`transform.yaml` config

The `.utlx` source and schemas are identical across environments. Only the `transform.yaml` varies (or the runtime override via the Admin API).

10.8 The Full Cycle

A commit to `main` triggers the pipeline. Within two minutes:

1. The bundle is built from the repository.
2. It is validated against the running UTLXe (dry run — does it compile?).
3. It is deployed (atomic replacement of all transformations).
4. Each transformation is tested with sample input.
5. The pipeline reports success or failure.

From commit to verified production deployment in under two minutes, with no custom Docker images, no container restarts, and no downtime.

11 Troubleshooting

This chapter lists the most common problems, their symptoms, and how to fix them.

11.1 Container Starts But `ready=false`

Symptom: Health endpoint returns `{"status":"UP", "transformations":0, "ready":false}`. No traffic is routed.

Cause: No transformations are loaded. The container started empty.

Fix:

- If using persistent storage: check that the Azure File Share is mounted correctly. Run `az containerapp show` and verify the volume mount configuration.
- If using CI/CD re-deploy: check that the pipeline ran and uploaded the bundle.
- Manual fix: upload a transformation via the Admin API.

11.2 Transformation Upload Fails (400)

Symptom: `POST /admin/transformations/{name}` returns 400 with an error message.

Cause: Syntax error in the `.utlx` source. The compilation failed.

Fix: Read the error response — it includes the line number and a description of the problem.

```
{
  "status": "rejected",
  "errors": [
    {"transformation": "invoice", "line": 14, "message": "Unknown function: concatX"}
  ]
}
```

Test locally before uploading: `echo '{}' | utlx -e 'your expression'` catches syntax errors immediately.

11.3 Messages Failing (500 on Data Plane)

Symptom: The data plane returns 500 for some or all messages. Error count increases in Prometheus.

Cause: Runtime error in the transformation — null reference, type mismatch, missing field.

Diagnose:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/errors
```

This shows the last 100 errors with the input that caused each failure.

Fix: Upload a corrected transformation. Test with the failing input before resuming:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"the":"failing input"}' \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/test
```

11.4 High Latency

Symptom: p99 latency exceeds 100ms or response times are inconsistent.

Cause 1 — Large messages: Parsing and serialization dominate for messages above 50 KB.

Check average message size and consider whether the transformation can be simplified, or upgrade to the Professional plan for more memory.

Cause 2 — GC pressure: The heap is too small for the workload.

Check `utlxe_heap_used_bytes` in Prometheus. If it stays above 80% of the max, upgrade the plan.

Cause 3 — Swap: The container has swap enabled.

Fix: set memory request equal to memory limit in the Container App configuration. See Chapter 7 for details.

11.5 Out of Memory (OOM Kill)

Symptom: Container restarts unexpectedly. Container logs show `Killed` or the exit code is 137.

Cause: A message (or burst of messages) consumed more heap than available. The JVM exceeded the container memory limit, and Kubernetes killed the process.

Fix:

- Set `maxInputSize` to reject oversized messages before parsing.
- Upgrade to a larger plan.
- Check for transformations that create large intermediate objects (deeply nested `map` of `map` operations).

11.6 Admin API Returns 403

Symptom: All admin endpoints return 403 Forbidden.

Cause 1: `UTLXE_ADMIN_KEY` is not set. When no key is configured, all admin endpoints are locked.

Cause 2: The `X-Admin-Key` header value does not match the environment variable.

Fix: Verify the key:

```
# Check if the env var is set
az containerapp show -n utlxe -g myResourceGroup \
  --query "properties.template.containers[0].env"
```

11.7 Transformations Lost After Restart

Symptom: After a container restart, health shows `transformations: 0`.

Cause: No persistent storage is configured. The container filesystem is ephemeral.

Fix:

- Enable persistent storage (Azure Files volume mount) by redeploying with the "Enable persistent transformation storage" option.
- Or set up a CI/CD pipeline to re-deploy the bundle after each container start.

11.8 Dapr Not Delivering Messages

Symptom: Messages are in the Service Bus queue but UTLXe is not processing them.

Cause 1 — Not ready: UTLXe health shows `ready: false`. Dapr waits for the app to be ready before delivering messages.

Fix: Upload transformations. Dapr begins delivering once `ready: true`.

Cause 2 — Binding name mismatch: The Dapr component name does not match a transformation name, and no `X-UTLXe-Transform` header is configured.

Fix: Verify that the Dapr component `metadata.name` matches the transformation name, or configure the transform header override.

Cause 3 — Transformation is paused: UTLXe returns 503, Dapr does not acknowledge.

Fix: Resume the transformation:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/{name}/resume
```

11.9 Schema Validation Failing Unexpectedly

Symptom: Messages that used to work are now rejected by validation.

Cause: The schema was updated but the incoming messages still use the old format.

Quick fix: Disable validation temporarily:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"policy":"off"}' \
  http://<admin>:8081/admin/transformations/{name}/validation
```

Proper fix: Update the transformation to handle both the old and new message formats, or coordinate the schema change with the message producer.

11.10 Diagnostic Commands Reference

```
# Health and readiness
curl http://<internal>:8081/health

# Engine info (version, uptime, config)
curl -H "X-Admin-Key: $KEY" http://<internal>:8081/admin/info

# List transformations with status
curl -H "X-Admin-Key: $KEY" http://<internal>:8081/admin/transformations

# Recent errors for a transformation
curl -H "X-Admin-Key: $KEY" \
  http://<internal>:8081/admin/transformations/{name}/errors

# Test a transformation
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"test":"data"}' \
  http://<internal>:8081/admin/transformations/{name}/test

# Effective validation state
curl -H "X-Admin-Key: $KEY" \
  http://<internal>:8081/admin/transformations/{name}/validation

# Container logs
az containerapp logs show -n utlxe -g myResourceGroup --follow
```

Appendices

Reference Material

12 Appendix A: API Reference

This appendix lists every endpoint on both ports with request format, response format, and status codes.

12.1 Authentication

All `/admin/*` endpoints require the header `X-Admin-Key: {value}`. The value must match the `UTLXE_ADMIN_KEY` environment variable. If the variable is not set, all admin endpoints return 403.

The `/health`, `/metrics`, and `/api/*` endpoints do not require authentication.

12.2 Admin Port (8081)

12.2.1 Bundle Endpoints

Method	Path	Description
`POST`	`/admin/bundle`	Upload a `.zip` bundle. Replaces all transformations and schemas atomically.
`GET`	`/admin/bundle`	Export the current state as a downloadable `.zip`.
`DELETE`	`/admin/bundle`	Remove all transformations and schemas. Returns to empty state.
`POST`	`/admin/bundle/validate`	Upload and validate without deploying (dry run).

12.2.2 Transformation Endpoints

Method	Path	Description
`GET`	`/admin/transformations`	List all deployed transformations with status and metrics.
`GET`	`/admin/transformations/{name}`	Get details: config, compile status, metrics, source.
`POST`	`/admin/transformations/{name}`	Deploy or update. Accepts `multipart/form-data` with `source` (required) and `config` (optional).
`POST`	`/admin/transformations/{name}/config`	Update config only. No recompile.
`DELETE`	`/admin/transformations/{name}`	Remove a single transformation.

12.2.3 Testing Endpoint

Method	Path	Description
`POST`	`/admin/transformations/{name}/test`	Execute with sample input. Returns output or error. Not counted in metrics.

12.2.4 Operational Endpoints

Method	Path	Description
`GET`	`/admin/info`	Engine version, uptime, config, persistence mode, transformation count.
`POST`	`/admin/transformations/{name}/pause`	Stop processing. Data plane returns 503. Transformation stays compiled.
`POST`	`/admin/transformations/{name}/resume`	Resume processing.
`GET`	`/admin/transformations/{name}/errors`	Recent errors (ring buffer, last 100). Accepts `?limit=N`.
`GET`	`/admin/config`	View current engine configuration.

POST	/admin/config	Update runtime-safe configuration fields.

12.2.5 Validation Override Endpoints

Method	Path	Description
GET	/admin/transformations/{name}/validation	Get effective validation state (policy, source, config default).
POST	/admin/transformations/{name}/validation	Set a runtime override. Body: {"policy":"off"}. Ephemeral — not persisted.
DELETE	/admin/transformations/{name}/validation	Remove override. Revert to config or header default.

12.2.6 Schema Endpoints

Method	Path	Description
GET	/admin/schemas	List all uploaded schemas with filename and size.
GET	/admin/schemas/{filename}	Download a schema file.
POST	/admin/schemas/{filename}	Upload or replace a schema. Accepts multipart/form-data with file.
DELETE	/admin/schemas/{filename}	Remove a schema.

12.2.7 Health Endpoints (no authentication)

Method	Path	Description
GET	/health	Returns status, transformation count, and ready flag.
GET	/metrics	Prometheus metrics in text exposition format.

12.3 Data Plane Port (8085)

Method	Path	Description
GET	/api/transformations	List available transformations (discovery). No authentication.
POST	/api/transform/{name}	Execute a transformation. Body is the input message. Content-Type header determines format.

12.4 Response Status Codes

Code	Meaning
200	Success.
400	Bad request — compilation error, invalid input, malformed ZIP.
403	Forbidden — missing or wrong X-Admin-Key, or key not configured.
404	Transformation not found.
413	Message too large — exceeds maxInputSize.
500	Transformation runtime error (null reference, type mismatch, etc.).
503	Transformation is paused.

13 Appendix B: Configuration Reference

13.1 Environment Variables

Variable	Default	Description
`UTLXE_ADMIN_KEY`	*(none)*	Admin API authentication key. Required — if not set, all admin endpoints return 403.
`UTLXE_HEAP_SIZE`	`1536m`	JVM heap size. Starter: `1536m`. Professional: `3072m`.
`JAVA_OPTS`	*(see below)*	Additional JVM options. Default includes G1GC, container support, AlwaysPreTouch.

Default `JAVA_OPTS`:

```
-XX:+UseG1GC -XX:MaxGCPauseMillis=200 -XX:+UseContainerSupport -XX:+AlwaysPreTouch
```

13.2 Engine Configuration (engine.yaml)

The engine configuration file is optional. If present in the bundle, it overrides the defaults.

Field	Default	Description
`maxInputSize`	`5MB`	Maximum message size before rejection. Messages larger than this are rejected with 413.
`workers`	CPU cores	Worker thread pool size.
`healthPort`	`8081`	Health + admin API port.
`dataPort`	`8085`	Data plane port.

13.3 Transformation Configuration (transform.yaml)

Per-transformation configuration. Optional — if absent, defaults apply.

Field	Default	Description
`strategy`	`COMPILED`	Execution strategy: `TEMPLATE`, `COPY`, `COMPILED`, `AUTO`.
`validationPolicy`	`strict`	Input validation: `strict` (reject), `warn` (log), `off` (skip).
`maxConcurrent`	*(unlimited)*	Maximum concurrent executions for this transformation.
`outputBinding`	*(none)*	Dapr output binding name for sending transformed messages.

Inputs and schemas:

```yaml
strategy: COMPILED
validationPolicy: strict
maxConcurrent: 4
outputBinding: orders-out
inputs:
  - name: input
    schema: order.xsd
output:
  schema: invoice.xsd
```

13.4 JVM Tuning Reference

Flag	Purpose

`-XX:+UseG1GC`	G1 garbage collector. Good for large heaps with pause-time targets.
`-XX:MaxGCPauseMillis=200`	Target maximum GC pause. G1 adjusts its behavior to meet this target.
`-XX:+UseContainerSupport`	Respect cgroup memory limits instead of reading host memory.
`-XX:+AlwaysPreTouch`	Allocate the entire heap at startup. Fail fast if not enough RAM.
`-Xmx${UTLXE_HEAP_SIZE}`	Maximum heap size. Set via `UTLXE_HEAP_SIZE` environment variable.

13.5 Container Plans

Plan	Container RAM	Heap	Max message	vCPU
Starter	2 GB	1536 MB	50 KB	1–2
Professional	4 GB	3072 MB	200 KB	2–4

The heap is set to 75% of container memory. The remaining 25% covers JVM metaspace, thread stacks, native memory, and the operating system.

13.6 Bundle ZIP Format

```
bundle.zip
  schemas/                    (optional)
    order.xsd
    invoice.json
  transformations/
    {name}/
      {name}.utlx             (required)
      transform.yaml          (optional)
  engine.yaml                 (optional)
```

13.7 Data Directory Layout

The on-disk state under `/utlxe/data/`. Written by the Admin API, read at startup.

```
/utlxe/data/
  schemas/
    order.xsd
    invoice.json
  transformations/
    invoice-to-ubl/
      invoice-to-ubl.utlx
    order-enrichment/
      order-enrichment.utlx
```

If Azure Files is mounted at `/utlxe/data/`, this directory survives container restarts.

14 Appendix C: UTL-X Quick Reference

A syntax cheat sheet for the most common transformation patterns. For the complete language reference, see the companion book *UTL-X: One Language, All Formats*.

14.1 File Structure

```
%utlx 1.0
input json
output json
---
{ transformation body }
```

14.2 Format Headers

`input json`	JSON input
`input xml`	XML input
`input csv {header: true}`	CSV with header row
`input yaml`	YAML input
`output xml`	XML output
`input json {schema: "f.xsd"}`	With schema validation

14.3 Property Access

`$input.name`	Dot notation
`$input.items[0]`	Array index (0-based)
`$input.@id`	XML attribute
`$input..name`	Recursive descent

14.4 Object Construction

```
{key: value, other: value}
{name: $input.n, ...$input}     // spread operator
```

14.5 Let Bindings

```
let x = expression
```

Multiple bindings separated by newlines or commas.

14.6 Conditionals

```
if (condition) valueA else valueB

match value {
  "A" -> resultA,
  "B" -> resultB,
  _ -> defaultResult
}
```

14.7 Array Operations

`map(arr, (x) -> expr)`	Transform each element
`filter(arr, (x) -> bool)`	Keep matching elements
`reduce(arr, init, (acc, x) -> expr)`	Aggregate to single value
`find(arr, (x) -> bool)`	First matching element
`sortBy(arr, (x) -> key)`	Sort by key function
`groupBy(arr, (x) -> key)`	Group by key function
`flatMap(arr, (x) -> arr)`	Map and flatten
`size(arr)`	Array length
`first(arr)` / `last(arr)`	First or last element

14.8 Pipe Operator

```
$input.items
  | filter(_, (x) -> x.active)
  | map(_, (x) -> x.name)
  | sortBy(_, (x) -> x)
```

The underscore _ is a placeholder for the piped value.

14.9 String Functions

`concat(a, b, c)`	Concatenate
`upperCase(s)` / `lowerCase(s)`	Case conversion
`trim(s)`	Remove whitespace
`replace(s, old, new)`	Replace substring
`contains(s, sub)`	Check substring
`substring(s, start, end)`	Extract substring
`length(s)`	String length
`split(s, delimiter)`	Split to array
`startsWith(s, prefix)` / `endsWith(s, suffix)`	Check prefix or suffix

14.10 Math Functions

`round(n, decimals)`	Round to decimal places
`floor(n)` / `ceil(n)`	Floor or ceiling
`abs(n)`	Absolute value
`min(a, b)` / `max(a, b)`	Minimum or maximum

14.11 Type Functions

`toString(v)` / `toNumber(v)` / `toBoolean(v)`	Type conversion
`isNull(v)`	Null check
`typeOf(v)`	Type name as string

14.12 Date Functions

`now()`	Current timestamp
`formatDate(d, pattern)`	Format a date (e.g., `"yyyy-MM-dd"`)
`parseDate(s, pattern)`	Parse a date string

14.13 Security Functions

`sha256(s)` / `sha512(s)`	Cryptographic hash
`hmacSHA256(s, key)`	HMAC signature
`encryptAES(data, key)` / `decryptAES(data, key)`	AES encryption
`base64Encode(s)` / `base64Decode(s)`	Base64 encoding
`mask(s, start, end, char)`	Mask sensitive data
`env("VAR_NAME")`	Read environment variable (engine only)

14.14 User-Defined Functions

```
function name(param1, param2) {
  expression
}
```

Defined before the main transformation body. Can call standard library functions and other user-defined functions.